COE 718 Final Project: Media Center Final Report

Jameson Stanley Cruz — Student ID: 501057427

December 1, 2024

Contents

1 Abstract **2**

2 Introduction **2**

3 System Design **2**

4 Completed Work **5**

5 Conclusion **14**

6 References **14**

7 Appendix **14**

Listings

1 Main Menu Navigation Code 3
2 2D Array for Level 1 4
3 usbdmain.c 14
4 photos.c 17
5 MP3.c 18
6 game.c 22
7 game.h 27

List of Figures

1 Project Design Navigation Flowchart 4
2 Media Center Main Menu 6
3 Photo Gallery 6
4 MP3 Player 7
5 Sokoban Game Start Screen 7
6 Sokoban Game Level 1 8
7 Sokoban Game Level 2 8
8 Illegal Move Alert 1 9
9 Illegal Move Alert 2 10
10 Illegal Move Alert 3 11
11 Illegal Move Alert 4 12
12 Level Up Screen 13
13 Win Screen 13

1 Abstract

The objective of this final project is to implement a media center using the MCB1700 board and uVision software, along with all the programming concepts that have been taught and used throughout the previous labs. The features of the media center include a photo gallery which displays .bmp files onto the LCD, an MP3 player capable of streaming MP3 files from the PC onto the MCB1700 board speaker, and a game center with an animated game on the LCD which supports joystick controls. In addition, a menu which allows the user to display and execute its features will be implemented as well.

2 Introduction

The main menu is the initial display on the LCD upon loading the program onto the MCB1700 board. The title of the project, along with my credentials, will be displayed at the top of the main menu. The three feature options that include the photo gallery, the MP3 player, and the game will also be displayed on the menu below the title. The current option hovered over by the joystick input will be highlighted with a blue background on the LCD display. When an option is selected in the main menu, the selected task will be executed through a function call similar to using inter-thread communication signals implemented in previous labs.

The photo gallery has a title at the top of the display similar to the main menu, as well as a message at the bottom that informs the user of the controls of the photo gallery. The user will be able to toggle through several photos in the gallery back and forth by pressing up or down on the joystick. The photos included in the photo gallery are also the same photos that will be used in the game to save memory usage for the program. The user will also be able to return to the main menu by pressing select on the joystick.

The MP3 player also has a title at the top of the display similar to the main menu, as well as a message at the bottom informing the user of the controls of the MP3 player. The user will be able to adjust the volume of the audio that is played by rotating the potentiometer on the MCB1700 board. Rotating it counterclockwise should increase the volume and vice versa. The user can return to the main menu by pressing select on the joystick.

The game that has been implemented is a sokoban puzzle game. The main objective of the game is to push all the boulders on the map to the designated targets, which are the holes. The main challenge of the game is that the player can only move one tile at a time while pushing the boulder in front of them if there is a free space (no pulling) and the player cannot push two boulders at once. In addition, the player must navigate the boulders through a maze-like map with rocks in the way which will make you think twice about the next move you'll make. The level is complete when all the holes have been filled with boulders.

For the media center, two levels have been designed for the game. The game has a start screen upon selection and transitions into the first level upon pressing select on the joystick. Once a level is completed, the user presses UP on the joystick to move on to the next level. Once all levels have been completed, the user is shown a win screen and can go back to the main menu by pressing UP on the joystick.

3 System Design

The main hardware components of the MCB 1700 board that will be used in the project's design include the LCD, the joystick, the speakers, the LEDs and the potentiometer. The main two hardware components that will be focused on throughout the media center's implementation are the LCD for the graphics display and the joystick to integrate user input. The speaker and potentiometer are exclusively used in the MP3 player, therefore there will be less emphasis on their implementation on the software components. The LEDs flash depending on the feature that is currently being used. Led 0 is on when the photo gallery is in use, led 1 is on when the MP3 player is in use and led 2 is on when the game is in use. In addition, for the sokoban game, different LEDs are activated whenever the player makes an illegal move.

A noteworthy thing to mention about the hardware component of the MCB 1700 board is the size of the LCD screen. The MCB 1700 board features a 320 x 240 screen resolution, therefore the size limitation plays a factor in the design of the photo gallery and the game, since both of them use imported pictures of

varying sizes. The photos in the photo gallery were set to a size of 120 x 120 to save memory space and to leave space for menu screen at the top and bottom.

The photos in the sokoban game were set to a size of 20 x 20. Since sokoban is a game that uses a 2D grid as a map, the size for a single tile should maximize the size of the grid while still keeping the size large enough so that it wouldn't look too small on the screen. By setting each tile in the 2D grid to a size of 20 x 20, this allowed for the space to design sokoban levels that are of size 12 x 16 on the MCB 1700 board.

The software components will mainly consist of implementing the joystick's input to display the desired output on the LCD.

For the menu, a switch statement is used in a similar fashion as lab 1, which uses the joystick input given from the 'getbutton()' function call and calls a certain thread function based on the selected option on the menu.

Listing 1: Main Menu Navigation Code

```
KBD_but = get_button ();
switch(KBD_but){
    case KBD_SELECT:
            GLCD_Clear(White);
                switch (disp_val){
                 case 1:
                      LED_On(0);
                      photos();
                      break;
                 case 2:
                      LED_On(1);
                      MP3();
                      break;
                 case 3:
                      LED_On(2);
                      game();
                      break;
                }
          break;
          case KBD_UP:
              if(disp_val > 1)
                  disp_val--;
              else
                  disp_val = 3;
          break;
          case KBD_DOWN:
              if(disp_val < 3)
                  disp_val++;
              else
                  disp_val = 1;
          break;
}
```

In order to determine the current position of the option being selected on the menu, a separate switch statement is used to increment and decrement an int variable called 'dispval' when the user inputs UP or DOWN on the joystick, respectively. If SELECT is pressed on the joystick, the user is sent to one of the three functions based on the current value of 'dispval' as shown in Figure 1.

The photo gallery uses similar switch statements to invoke certain functions based on the joystick input. The file names of the photos in the photo gallery are stored in an array named 'gallery' and upon pressing UP or DOWN on the joystick, an int variable called 'pic' is incremented or decremented, respectively, and the array index corresponding to the value of the int variable will be outputted on the LCD display. By pressing SELECT, led 0 is turned off, indicating that the media center is no longer using the photo gallery,

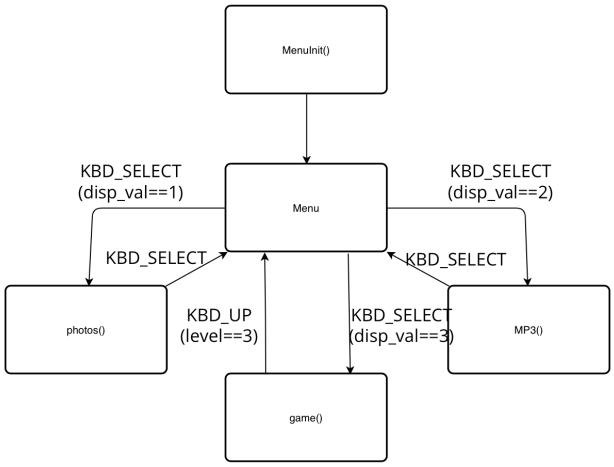

Figure 1: Project Design Navigation Flowchart

and the user is sent back to the main menu. The executable c file can be found in the appendix with the name photos.c.

The MP3 player's implementation is mostly given in the provided 'USBAudio' project example, with the addition of a simple joystick function call, which yields the function back to the menu when the select button is pressed on the joystick and turns off led 1. The executable c file can be found in the appendix with the name MP3.c.

The starting game screen only appears once at the beginning when the game() function is called from the main menu. Similar to the previous two features, the game uses switch statements to invoke certain functions based on the joystick input. By pressing SELECT, this invokes the function initBoard() which sets the board of the game to its initial position based on the current level. Pressing either UP, DOWN, LEFT or RIGHT on the joystick invokes the movePlayer() function which handles player movement in the game. There is also an if statement which checks whether the current level has been complete. By pressing UP, the player is sent to the next level by invoking the initBoard() function but if the player has cleared both levels, the player is sent back to the main menu and led 2 is turned off.

The initBoard() function uses nested for loops to copy a 2D array provided in the game.h file (found in the appendix) onto the 2D int array variable called 'board' and declares the player's starting position depending on the current level. The initial board settings and player's starting position are changed for each level if 'DEMO' is uncommented at the beginning of the code and is only used for demo purposes. In addition, there are some instances in the sokoban game where the puzzle becomes unsolvable such as when a boulder is stuck in a position where it cannot be pushed anywhere. To solve this problem, the user has the choice to click the SELECT button anytime to reset the board back to its initial position. To output the contents of the 2D array on the LCD, the printBoard() function is called at the end of the function.

Before moving on to the printBoard() function, it is important to understand how the 2D arrays were implemented in the first place. The photos used for the game are stored in an array similar to the photo gallery, with their array indices spanning from 0-5. Based on the order of the 'gallery' in game.h (found in the appendix), 0 is an empty square, 1 is the player, 2 is a boulder, 3 is a rock (or obstacle), 4 is a hole (target) and 5 is a boulder inside a hole. To design a level, a 12 x 16 int array can be initialized with integers from 0 to 5 depending on which object you wish to create at that specific coordinate. For instance, if there is a 3 on the 3rd row and 4th column of the array, then a rock would appear on the screen at that specific coordinate. An example of this can be observed below. The output of this array can be observed in Figure 6.

Listing 2: 2D Array for Level 1

```
//0=empty, 1=player, 2=boulder, 3=rock, 4=hole, 5=rock_in
extern int level1[ROWS][COLS] = {
        {0, 0, 0, 0, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0},
```

```
        {0, 0, 0, 0, 3, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 3, 2, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 3, 3, 3, 0, 0, 2, 3, 3, 3, 0, 0, 0, 0, 0},
        {0, 0, 3, 0, 0, 2, 0, 0, 2, 0, 3, 0, 3, 3, 3, 3},
        {3, 3, 3, 0, 3, 0, 3, 3, 3, 0, 3, 0, 3, 0, 0, 4},
        {3, 0, 0, 0, 3, 0, 3, 3, 3, 0, 3, 3, 4, 0, 0, 4},
        {3, 0, 2, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4},
        {3, 3, 3, 3, 3, 3, 0, 3, 3, 3, 0, 3, 1, 0, 0, 4},
        {0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 3, 3, 0, 0, 4},
        {0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
};      ;
```

The printBoard() function is the implementation of what was explained above. Using a nested for loop, the pictures from the gallery are outputted onto the LCD one by one based on the current values of the 'board' variable, which stores the current state of the game board.

The movePlayer() function takes parameters dx and dy, which are either positive or negative 1 based on the user's input and increments them with the player's current x and y position. These new values are stored as temporary variables and a new variable called 'destination' determines the end position of the player after the inputted movement. Several conditional statements use the 'destination' variable to handle every possible scenario.

If the end position is empty, the current position is replaced with an empty square and the player moves to the new position.

If the end position is a boulder, which in this case would require the player to push the boulder, several other conditions need to be accounted for since two objects need to be moved at once. Temporary variables for the boulder's new x and y positions are created by incrementing the new position once again (2 spaces ahead), which are used to handle the different scenarios for both the boulder's and player's movement. If the next space is empty, simply replace the current position with an empty space and move both the player and the boulder one space in the specified direction. If the next space is a hole, then the boulder picture should be replaced with a boulder in hole picture to show that one of the targets has been filled.

Every other condition should result in an illegal move and as a result, should display an error message at the bottom and it does not update the board or the player's position. In addition, a certain LED should light up to specify the type of illegal move being made. If there is a rock, LED 3 lights up as observed in Figure 8. If there is another boulder, LED 4 lights up as observed in Figure9. If there is already another boulder in a hole, LED 5 lights up as observed in Figure10.

The last two if statements also handle illegal moves made by the player which do not involve the boulder's movement. If there is a hole in front of the player, LED 6 lights up as observed in Figure 11.

Lastly, the checkWin() function constantly checks the board with a nested for loop if there are any holes remaining within the current level. If there are no more holes found in the board, the level is complete and the function returns 1.

4 Completed Work

The main menu has been implemented and is able to transition to the photo gallery, the MP3 player and the game from the main menu.

The photo gallery functionalities, which include toggling back and forth through photos and returning to the main menu, have been fully implemented.

The MP3 player is also functional as the speaker device can be connected upon selecting the MP3 player option in the main menu and can turn off when the user goes back to the main menu.

The game is also fully functional, including the start screen, levels 1 and 2, illegal move alerts, level up screen and the win screen.

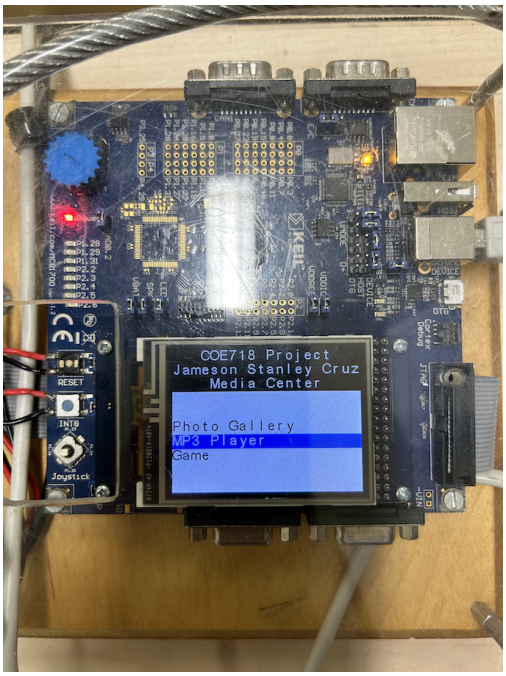

Figure 2: Media Center Main Menu

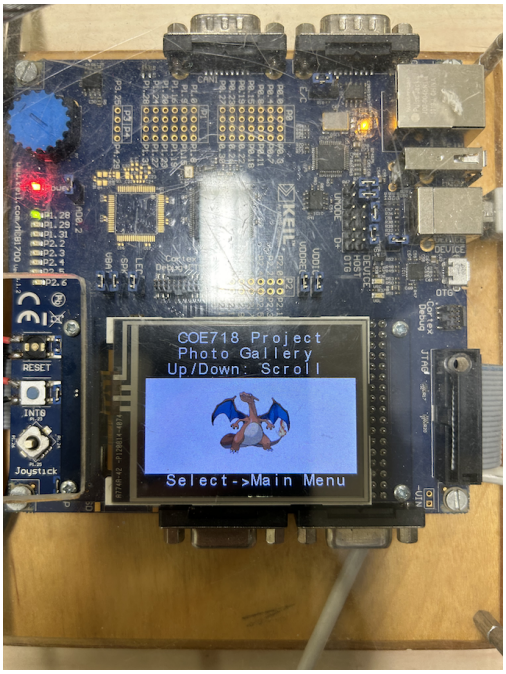

Figure 3: Photo Gallery

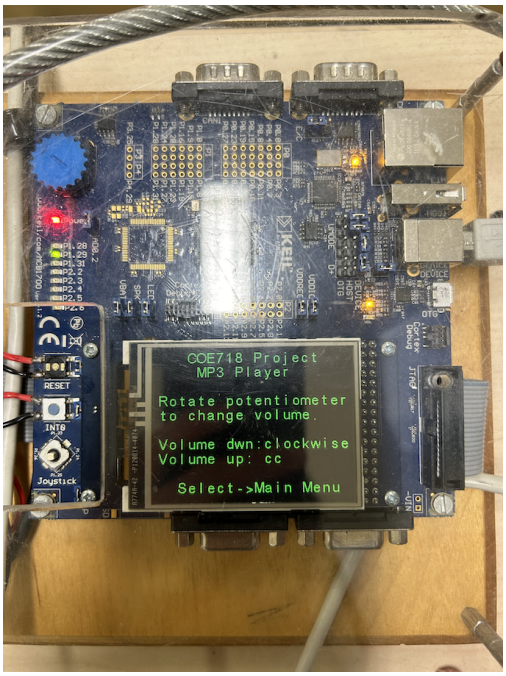

Figure 4: MP3 Player

Figure 5: Sokoban Game Start Screen

Figure 6: Sokoban Game Level 1

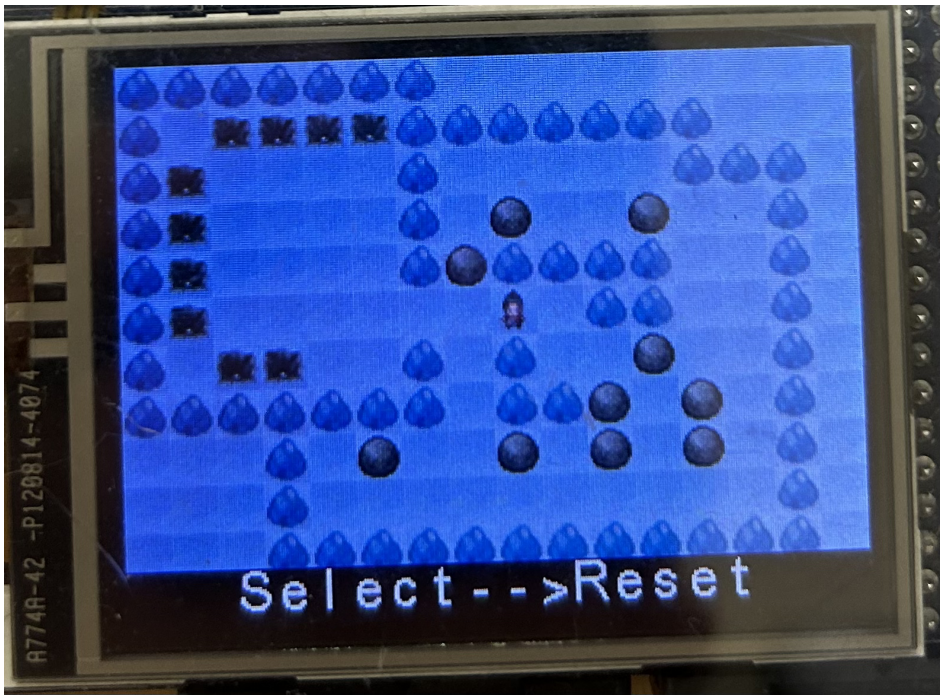

Figure 7: Sokoban Game Level 2

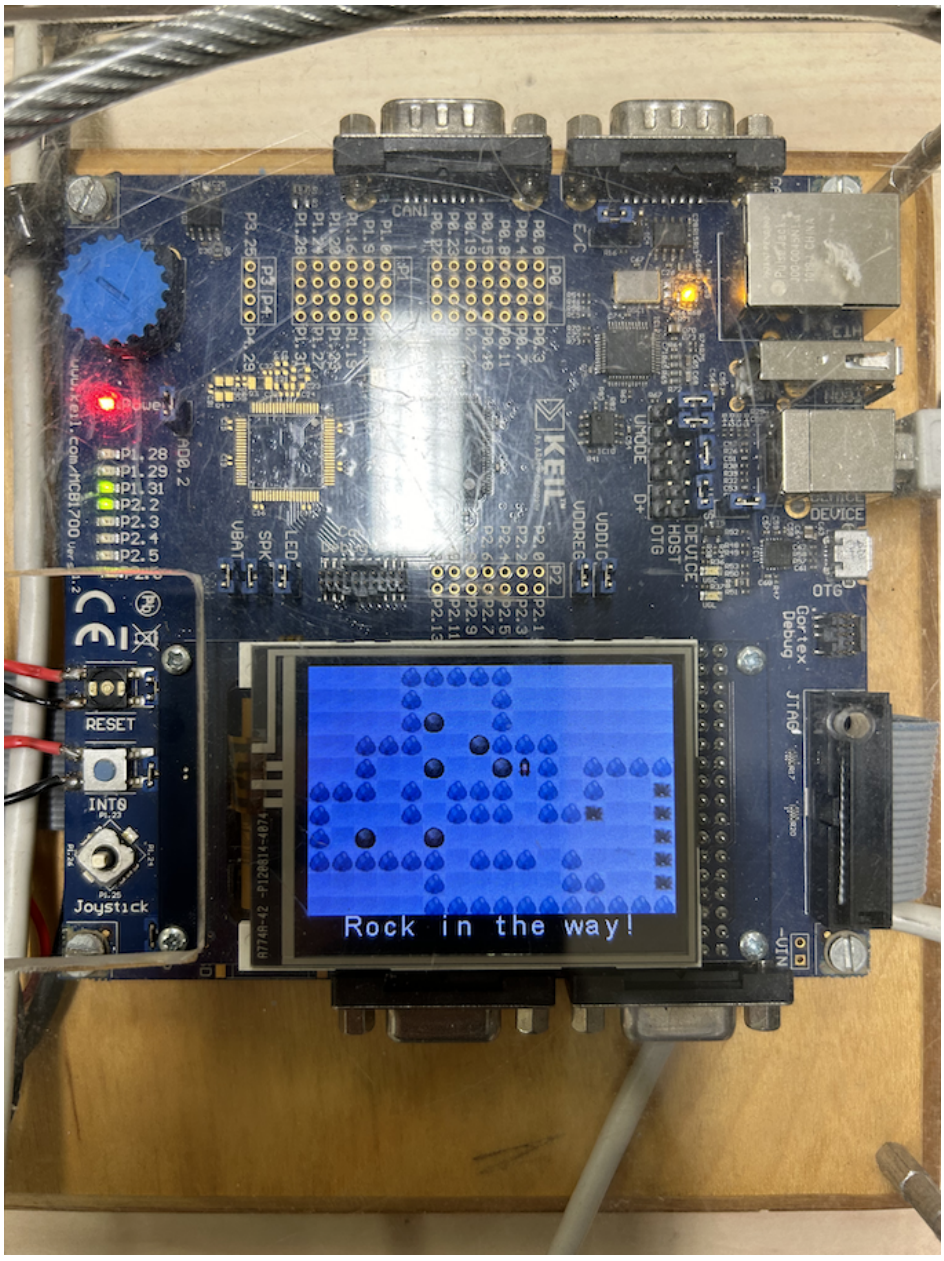

Figure 8: Illegal Move Alert 1

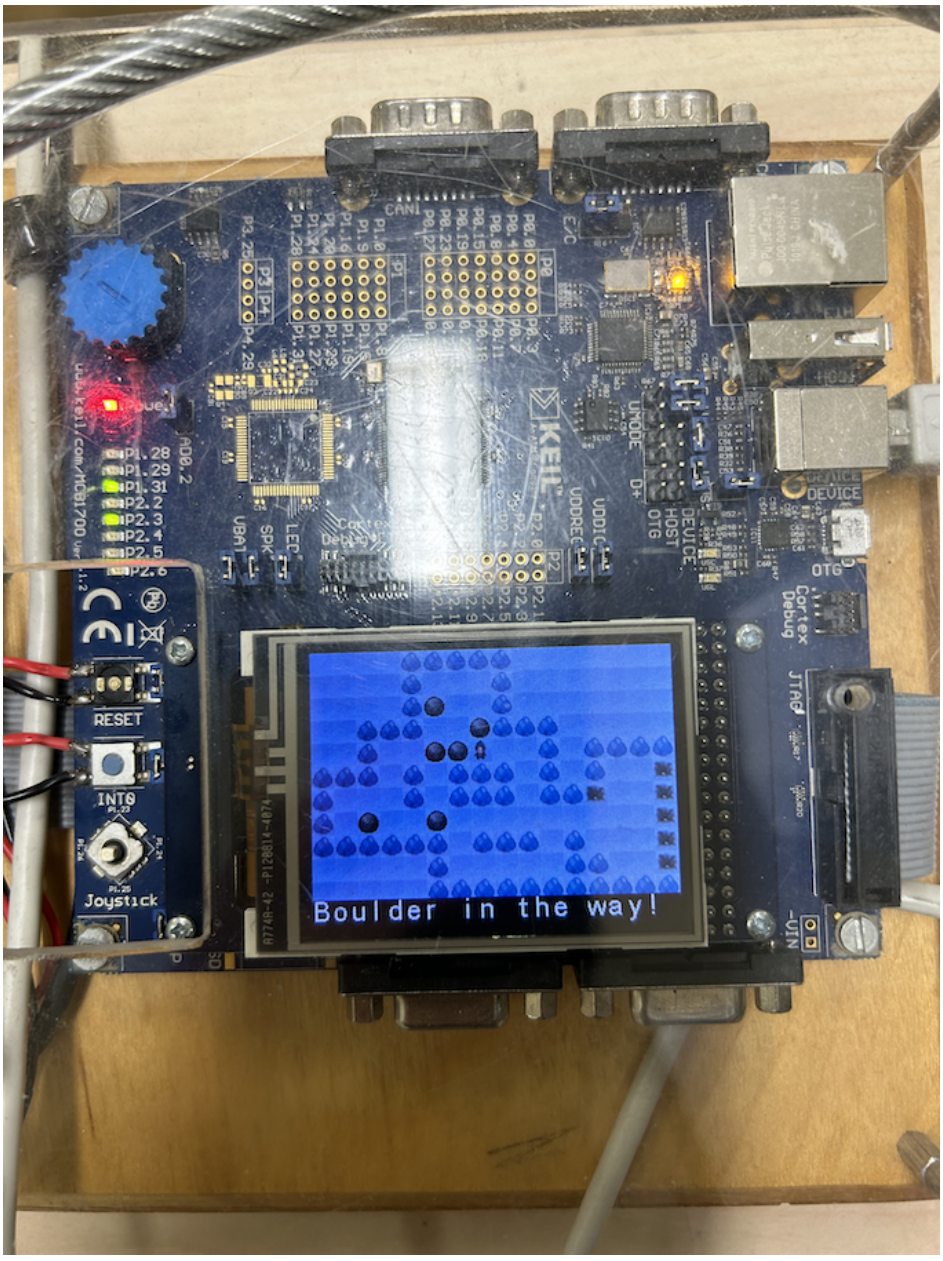

Figure 9: Illegal Move Alert 2

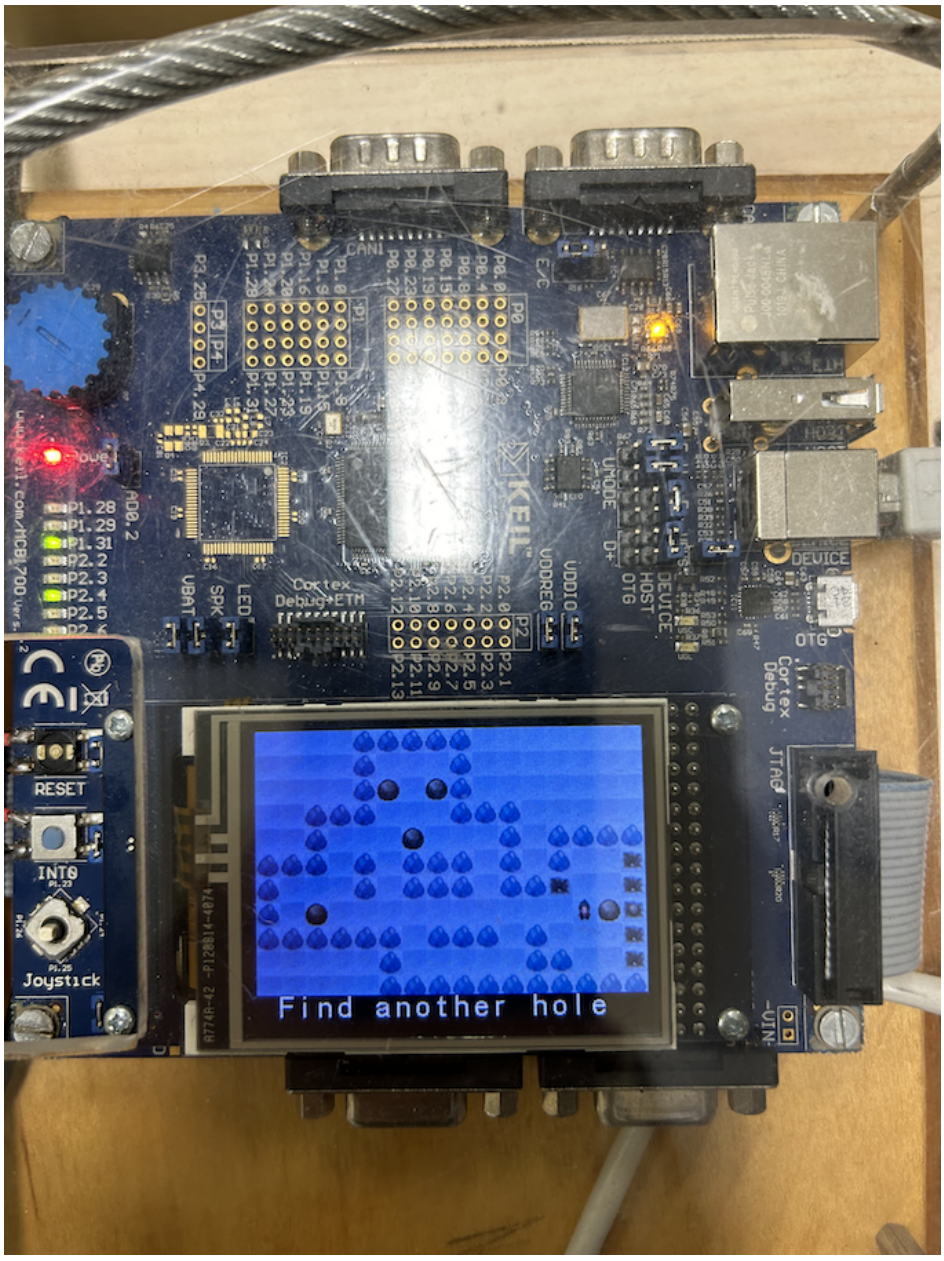

Figure 10: Illegal Move Alert 3

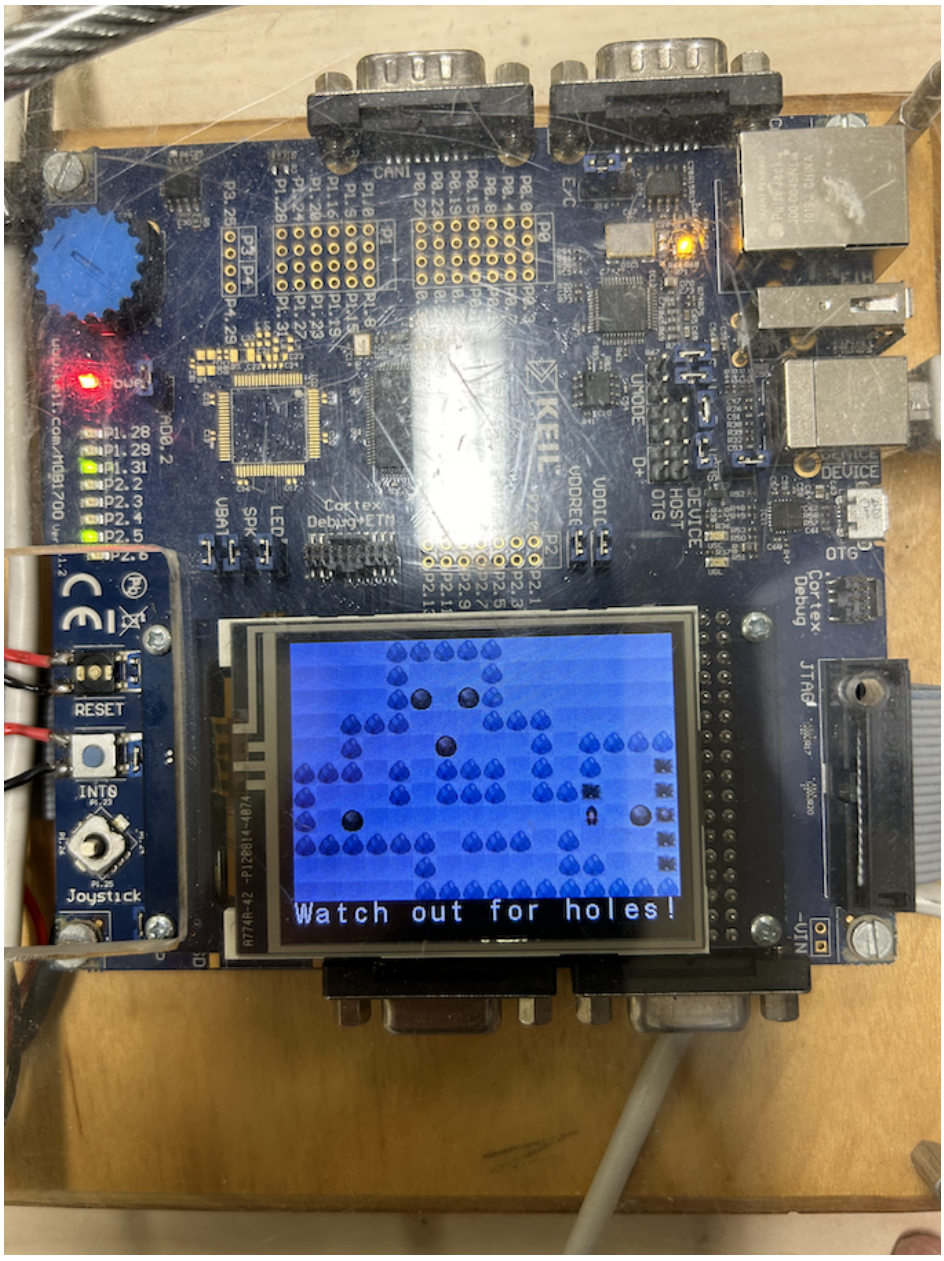

Figure 11: Illegal Move Alert 4

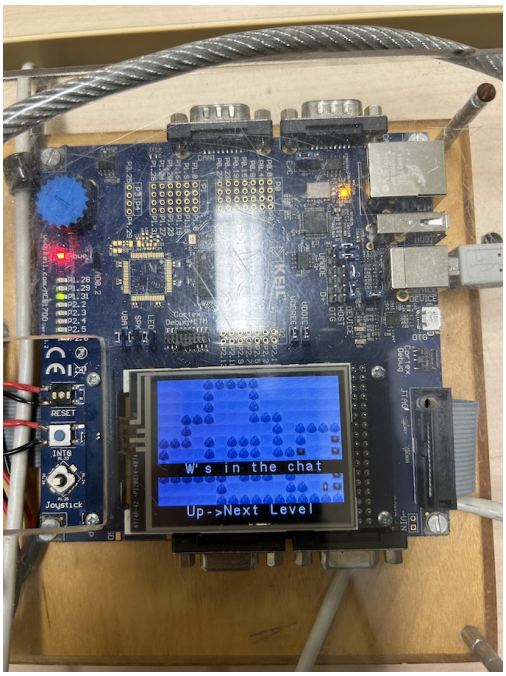

Figure 12: Level Up Screen

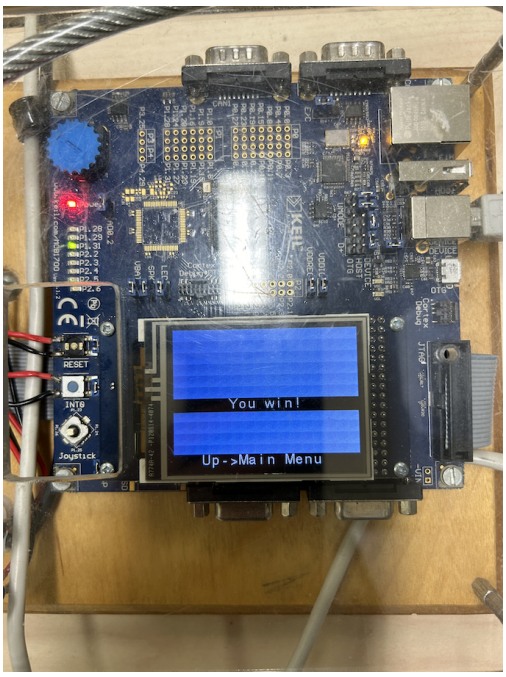

Figure 13: Win Screen

5 Conclusion

Overall, the project uses several components, such as hardware components with the MCB1700 board, as well as software components such as the use of uVision and programming concepts implemented throughout the previous labs. The software components included the given code for the joystick and the LCD display to name a few, which were used to implement the hardware component to support user and computer inputs. Furthermore, the sokoban game design is a culmination of many concepts learned throughout the past labs. Some instances include using the GLCDBitmap() function in a way that can properly display a 12 x 16 puzzle grid and turning on specific LEDs for different illegal moves. Currently, all three main features have been implemented completely and are functional for demo purposes.

6 References

Pierce, R. (2023). Sokoban. Math is Fun. https://www.mathsisfun.com/games/sokoban.html

7 Appendix

Listing 3: usbdmain.c

```
/*----------------------------------------------------------------------------
 * Name:    usbmain.c
 * Purpose: USB Audio Class Demo
 * Version: V1.20
 *----------------------------------------------------------------------------
 *      This software is supplied "AS IS" without any warranties, express,
 *      implied or statutory, including but not limited to the implied
 *      warranties of fitness for purpose, satisfactory quality and
 *      noninfringement. Keil extends you a royalty-free right to reproduce
 *      and distribute executable files created using this software for use
 *      on NXP Semiconductors LPC microcontroller devices only. Nothing else
 *      gives you the right to use this software.
 *
 * Copyright (c) 2009 Keil - An ARM Company. All rights reserved.
 *----------------------------------------------------------------------------*/

#include "LPC17xx.h"                        /* LPC17xx definitions */
#include "type.h"

#include "usb.h"
#include "usbcfg.h"
#include "usbhw.h"
#include "usbcore.h"
#include "usbaudio.h"

/*From LCD_Blinky Project*/
#include "string.h"
#include "GLCD.h"
#include "LED.h"
#include "KBD.h"

#define __FI        1                       /* Font index 16x24               */
/*
extern unsigned char ClockLEDOn;
```

```
extern unsigned char ClockLEDOff;
extern unsigned char ClockANI;
extern unsigned char Bg_16bpp_t[];
extern unsigned char Bg_16bpp_l[];
extern unsigned char Bg_16bpp_r[];
extern unsigned char Bg_16bpp_b[];
extern unsigned char ARM_Ani_16bpp[];
*/



/*****************************************************************************
**   Main Function  main()
*****************************************************************************/
int main (void)
{
                int KBD_but, disp_val = 0;
          LED_Init ();
                GLCD_Init();
                KBD_Init();

                GLCD_Clear(White);
        /*From LCD_Blinky Project*/
  for (;;) {                                /* Loop forever                                 */
        switch(disp_val){
                        case 0:
                                GLCD_SetBackColor(Black);
                                GLCD_SetTextColor(White);
                                GLCD_DisplayString(0, 0, __FI, "---COE718-Project---");
                                GLCD_DisplayString(1, 0, __FI, "Jameson-Stanley-Cruz");
                                GLCD_DisplayString(2, 0, __FI, "----Media-Center----");
                                GLCD_SetBackColor(White);
                                GLCD_SetTextColor(Black);
                                GLCD_DisplayString(5, 0, __FI, "Photo-Gallery-------");
                                GLCD_DisplayString(6, 0, __FI, "MP3-Player----------");
                                GLCD_DisplayString(7, 0, __FI, "Game----------------");
                        break;
                        case 1:
                                GLCD_SetBackColor(Black);
                                GLCD_SetTextColor(White);
                                GLCD_DisplayString(0, 0, __FI, "---COE718-Project---");
                                GLCD_DisplayString(1, 0, __FI, "Jameson-Stanley-Cruz");
                                GLCD_DisplayString(2, 0, __FI, "----Media-Center----");
                                GLCD_SetBackColor(Blue);
                                GLCD_SetTextColor(White);
                                GLCD_DisplayString(5, 0, __FI, "Photo-Gallery-------");
                                GLCD_SetBackColor(White);
                                GLCD_SetTextColor(Black);
                                GLCD_DisplayString(6, 0, __FI, "MP3-Player----------");
                                GLCD_DisplayString(7, 0, __FI, "Game----------------");
                        break;
                        case 2:
                                GLCD_SetBackColor(Black);
```

```
                GLCD_SetTextColor(White);
                GLCD_DisplayString(0, 0, __FI, "---COE718-Project---");
                GLCD_DisplayString(1, 0, __FI, "Jameson-Stanley-Cruz");
                GLCD_DisplayString(2, 0, __FI, "----Media-Center----");
                GLCD_SetBackColor(White);
                GLCD_SetTextColor(Black);
                GLCD_DisplayString(5, 0, __FI, "Photo-Gallery-------");
                GLCD_SetBackColor(Blue);
                GLCD_SetTextColor(White);
                GLCD_DisplayString(6, 0, __FI, "MP3-Player----------");
                GLCD_SetBackColor(White);
                GLCD_SetTextColor(Black);
                GLCD_DisplayString(7, 0, __FI, "Game----------------");
        break;
        case 3:
                GLCD_SetBackColor(Black);
                GLCD_SetTextColor(White);
                GLCD_DisplayString(0, 0, __FI, "---COE718-Project---");
                GLCD_DisplayString(1, 0, __FI, "Jameson-Stanley-Cruz");
                GLCD_DisplayString(2, 0, __FI, "----Media-Center----");
                GLCD_SetBackColor(White);
                GLCD_SetTextColor(Black);
                GLCD_DisplayString(5, 0, __FI, "Photo-Gallery-------");
                GLCD_DisplayString(6, 0, __FI, "MP3-Player----------");
                GLCD_SetBackColor(Blue);
                GLCD_SetTextColor(White);
                GLCD_DisplayString(7, 0, __FI, "Game----------------");
        break;
        case 6:
                GLCD_SetBackColor(Black);
                GLCD_SetTextColor(White);
                GLCD_DisplayString(0, 0, __FI, "---COE718-Project---");
                GLCD_DisplayString(1, 0, __FI, "-------Game---------");
        break;
}
KBD_but = get_button ();
        switch(KBD_but){
        case KBD_SELECT:
                GLCD_Clear(White);
                switch (disp_val){
                        case 1:
                                LED_On(0);
                                photos();
                        break;
                        case 2:
                                LED_On(1);
                                MP3();
                        break;
                        case 3:
                                LED_On(2);
                                game();
                        break;
                }
        break;
```

```
                        case KBD_UP:
                                if(disp_val > 1) {
                                        disp_val--;
                                } else{
                                        disp_val = 3;
                                }
                        break;
                        case KBD_DOWN:
                                if(disp_val < 3) {
                                        disp_val++;
                                } else {
                                        disp_val = 1;
                                }
                        break;
                }
  }
  /********* The main Function is an endless loop ***********/
  while( 1 );
}

/*****************************************************************************
**                            End Of File
*****************************************************************************/
```

Listing 4: photos.c

```
/*----------------------------------------------------------------------------
 * Name:    Blinky.c
 * Purpose: LED Flasher and Graphic Demo
 * Note(s):
 *----------------------------------------------------------------------------
 * This file is part of the uVision/ARM development tools.
 * This software may only be used under the terms of a valid, current,
 * end user licence from KEIL for a compatible version of KEIL software
 * development tools. Nothing else gives you the right to use this software.
 *
 * This software is supplied "AS IS" without warranties of any kind.
 *
 * Copyright (c) 2008-2011 Keil - An ARM Company. All rights reserved.
 *----------------------------------------------------------------------------*/

#include "LPC17xx.h"                        /* NXP LPC17xx definitions
*/
#include "string.h"
#include "GLCD.h"
#include "LED.h"
#include "KBD.h"
#include "greninja.c"
#include "charizard.c"
#include "torterra.c"

#define __FI        1                       /* Font index 16x24               */

void menu_Init (void) {
                GLCD_Clear(White);
```

```
                GLCD_SetBackColor(Black);
                GLCD_SetTextColor(White);
                GLCD_DisplayString(0, 0, __FI, "---COE718 Project---");
                GLCD_DisplayString(1, 0, __FI, "---Photo Gallery----");
                GLCD_DisplayString(2, 0, __FI, "--Up/Down: Scroll---");
                GLCD_DisplayString(9, 0, __FI, "--Select->Main Menu-");
}
/*----------------------------------------------------------------------------
  Main Program
 *----------------------------------------------------------------------------*/
int photos (void) {                             /* Main Program
*/
        int KBD_but, pic = 0;

        unsigned char* gallery[]= {(unsigned char *)&TORTERRA_pixel_data,
    (unsigned char *)&CHARIZARD_pixel_data,(unsigned char *)&GRENINJA_pixel_data };

  KBD_Init ();
        LED_Init ();
  GLCD_Init();
        menu_Init();
        for (;;){
                KBD_but=get_button();
                switch(KBD_but) {
                        case KBD_UP:
                                pic= (pic + 1) % 3;
                                menu_Init();
                                GLCD_Bitmap (  100,  80, 120,  120, gallery[pic]);
                                break;
                        case KBD_DOWN:
                                pic= (pic - 1 + 3) % 3;
                                menu_Init();
                                GLCD_Bitmap (  100,  80, 120,  120, gallery[pic]);
                                break;
                        case KBD_SELECT:
                                GLCD_Clear(White);
                                LED_Off(0);
                                return 0;
                }
  }
}
```

Listing 5: MP3.c

```
/*----------------------------------------------------------------------------
 * Name:    usbmain.c
 * Purpose: USB Audio Class Demo
 * Version: V1.20
 *----------------------------------------------------------------------------
 *      This software is supplied "AS IS" without any warranties, express,
 *      implied or statutory, including but not limited to the implied
 *      warranties of fitness for purpose, satisfactory quality and
 *      noninfringement. Keil extends you a royalty-free right to reproduce
 *      and distribute executable files created using this software for use
 *      on NXP Semiconductors LPC microcontroller devices only. Nothing else
```

```
 *       gives you the right to use this software.
 *
 * Copyright (c) 2009 Keil - An ARM Company. All rights reserved.
 *----------------------------------------------------------------------------*/

#include "LPC17xx.h"                        /* LPC17xx definitions */
#include "type.h"
#include "GLCD.h"
*/
#include "KBD.h"
*/

#include "usb.h"
#include "usbcfg.h"
#include "usbhw.h"
#include "usbcore.h"
#include "usbaudio.h"

#define __FI        1                       /* Font index 16x24               */

extern  void SystemClockUpdate(void);
extern uint32_t SystemFrequency;
uint8_t  Mute;                              /* Mute State */
uint32_t Volume;                            /* Volume Level */

#if USB_DMA
uint32_t *InfoBuf = (uint32_t *)(DMA_BUF_ADR);
short *DataBuf = (short *)(DMA_BUF_ADR + 4*P_C);
#else
uint32_t InfoBuf[P_C];
short DataBuf[B_S];                         /* Data Buffer */
#endif

uint16_t  DataOut;                          /* Data Out Index */
uint16_t  DataIn;                           /* Data In Index */

uint8_t   DataRun;                          /* Data Stream Run State */
uint16_t  PotVal;                           /* Potenciometer Value */
uint32_t  VUM;                              /* VU Meter */
uint32_t  Tick;                             /* Time Tick */


/*
 * Get Potenciometer Value
 */

void get_potval (void) {
  uint32_t val;

  LPC_ADC->CR |= 0x01000000;                /* Start A/D Conversion */
  do {
    val = LPC_ADC->GDR;                     /* Read A/D Data Register */
  } while ((val & 0x80000000) == 0);        /* Wait for end of A/D Conversion */
  LPC_ADC->CR &= ~0x01000000;               /* Stop A/D Conversion */
```

```
  PotVal = ((val >> 8) & 0xF8) +          /* Extract Potenciometer Value */
           ((val >> 7) & 0x08);
}


/*
 * Timer Counter 0 Interrupt Service Routine
 *   executed each 31.25us (32kHz frequency)
 */

void TIMER0_IRQHandler(void)
{
  long  val;
  uint32_t cnt;
        int KBD_but;

  if (DataRun) {                          /* Data Stream is running */
    val = DataBuf[DataOut];               /* Get Audio Sample */
    cnt = (DataIn - DataOut) & (B_S - 1); /* Buffer Data Count */
    if (cnt == (B_S - P_C*P_S)) {         /* Too much Data in Buffer */
      DataOut++;                          /* Skip one Sample */
    }
    if (cnt > (P_C*P_S)) {                /* Still enough Data in Buffer */
      DataOut++;                          /* Update Data Out Index */
    }
    DataOut &= B_S - 1;                   /* Adjust Buffer Out Index */
    if (val < 0) VUM -= val;              /* Accumulate Neg Value */
    else         VUM += val;              /* Accumulate Pos Value */
    val  *= Volume;                       /* Apply Volume Level */
    val >>= 16;                           /* Adjust Value */
    val  += 0x8000;                       /* Add Bias */
    val  &= 0xFFFF;                       /* Mask Value */
  } else {
    val = 0x8000;                         /* DAC Middle Point */
  }

  if (Mute) {
    val = 0x8000;                         /* DAC Middle Point */
  }

  LPC_DAC->CR = val & 0xFFC0;             /* Set Speaker Output */

  if ((Tick++ & 0x03FF) == 0) {           /* On every 1024th Tick */
    get_potval();                         /* Get Potenciometer Value */
    if (VolCur == 0x8000) {               /* Check for Minimum Level */
      Volume = 0;                         /* No Sound */
    } else {
      Volume = VolCur * PotVal;           /* Chained Volume Level */
    }
    val = VUM >> 20;                      /* Scale Accumulated Value */
    VUM = 0;                              /* Clear VUM */
    if (val > 7) val = 7;                 /* Limit Value */
  }
```

```
  LPC_TIM0->IR = 1;                          /* Clear Interrupt Flag */

        KBD_but=get_button();

        if (KBD_but == KBD_SELECT)
              NVIC_SystemReset();
}



/*****************************************************************************
**   Main Function  main()
*****************************************************************************/
void MP3 (void)
{
  volatile uint32_t pclkdiv, pclk;

        GLCD_Init();
                              GLCD_Clear(Black);
                              GLCD_SetBackColor(Black);
                              GLCD_SetTextColor(Green);
                              GLCD_DisplayString(0, 0, __FI, "---COE718 Project---");
                              GLCD_DisplayString(1, 0, __FI, "----MP3 Player------");
                              GLCD_DisplayString(3, 0, __FI, "Rotate potentiometer");
                              GLCD_DisplayString(4, 0, __FI, "to change volume.---");
                              GLCD_DisplayString(6, 0, __FI, "Volume dwn: clockwise");
                              GLCD_DisplayString(7, 0, __FI, "Volume up: cc-------");
                              GLCD_DisplayString(9, 0, __FI, "--Select->Main Menu-");

  /* SystemClockUpdate() updates the SystemFrequency variable */
  SystemClockUpdate();

  LPC_PINCON->PINSEL1 &=~((0x03<<18)|(0x03<<20));
  /* P0.25, A0.0, function 01, P0.26 AOUT, function 10 */
  LPC_PINCON->PINSEL1 |= ((0x01<<18)|(0x02<<20));

  /* Enable CLOCK into ADC controller */
  LPC_SC->PCONP |= (1 << 12);

  LPC_ADC->CR = 0x00200E04;                  /* ADC: 10-bit AIN2 @ 4MHz */
  LPC_DAC->CR = 0x00008000;                  /* DAC Output set to Middle Point */

  /* By default, the PCLKSELx value is zero, thus, the PCLK for
   all the peripherals is 1/4 of the SystemFrequency. */
  /* Bit 2~3 is for TIMER0 */
  pclkdiv = (LPC_SC->PCLKSEL0 >> 2) & 0x03;
  switch ( pclkdiv )
  {
        case 0x00:
        default:
          pclk = SystemFrequency/4;
        break;
        case 0x01:
          pclk = SystemFrequency;
```

```
        break;
        case 0x02:
          pclk = SystemFrequency/2;
        break;
        case 0x03:
          pclk = SystemFrequency/8;
        break;
    }

    LPC_TIM0->MR0 = pclk/DATA_FREQ - 1;    /* TC0 Match Value 0 */
    LPC_TIM0->MCR = 3;                     /* TCO Interrupt and Reset on MR0
    LPC_TIM0->TCR = 1;                     /* TC0 Enable */
    NVIC_EnableIRQ(TIMER0_IRQn);

    USB_Init();                            /* USB Initialization */
    USB_Connect(TRUE);               /* USB Connect */

    /********* The main Function is an endless loop ***********/
    while( 1 );
}

/*****************************************************************************
**                            End Of File
*****************************************************************************/
```

Listing 6: game.c

```
#include "LPC17xx.h"                        /* NXP LPC17xx definitions
*/
#include "string.h"
#include "GLCD.h"
#include "LED.h"
#include "KBD.h"

#include <stdlib.h>

#include "game.h"        //includes 2D arrays for each level

#include "empty.c"
#include "player.c"
#include "boulder.c"
#include "rock.c"
#include "hole.c"
#include "rock_in.c"

#define __FI        1                       /* Font index 16x24               */
//#define NO_DEMO       0
/* Uncomment to use the designed levels */
#define DEMO                    0
/* Uncomment for demo purposes */

#define ROWS 12
#define COLS 16

#define EMPTY           0
```

```
#define PLAYER  1
#define BOULDER 2
#define ROCK            3
#define HOLE            4
#define ROCK_IN 5

//Game Pictures
unsigned char* gallery[]= {(unsigned char *)&EMPTY_pixel_data,
(unsigned char *)&PLAYER_pixel_data, (unsigned char *)&BOULDER_pixel_data,
(unsigned char *)&ROCK_pixel_data,
(unsigned char *)&HOLE_pixel_data,
(unsigned char *)&ROCK_IN_pixel_data};

//0=empty, 1=player, 2=boulder, 3=rock, 4=hole, 5=rock_in


int board[ROWS][COLS];  // Define the game board
int playerX = 3, playerY = 4; // Initial player position
int level;      // Initial level

void printBoard() {
    int i =0, j=0;
    for (i = 0; i < COLS; i++) {
        for (j = 0; j < ROWS; j++) {
          GLCD_Bitmap (  i*20,  j*20, 20,  20, gallery[board[j][i]]);
        }
    }
    #ifdef DEMO
    GLCD_DisplayString(0, 0, __FI, "-Demo-Purposes-Only-");
    #endif
    GLCD_DisplayString(9, 0, __FI, "---Select->Reset---");
}

void initBoard() {
    int i=0, j=0;
    for (i = 0; i < ROWS; i++) {
        for (j = 0; j < COLS; j++) {
            board[i][j] = 0;
        }
    }
        if (level==0)
        level++;
        if (level==1) {
                #ifdef NO_DEMO
                playerX = 8, playerY = 12;
                for (i = 0; i < ROWS; i++) {
                        for (j = 0; j < COLS; j++) {
                                board[i][j] = level1[i][j];
        }
    }
                #endif
                #ifdef DEMO
                playerX = 3, playerY = 4;
                board[3][4]=PLAYER;
```

```
                board[3][5]=BOULDER;
                for (i=0;i<COLS;i++) {
                        if (i%2==0)
                                board[5][i]=ROCK;
                }
                board[3][9]=HOLE;
#endif
        }
        else if (level==2) {
                #ifdef NO_DEMO
                playerX = 5, playerY = 8;
                for (i = 0; i < ROWS; i++) {
                        for (j = 0; j < COLS; j++) {
                                board[i][j] = level2[i][j];
        }
    }
                #endif
                #ifdef DEMO
                playerX = 3, playerY = 4;
                board[3][4]=PLAYER;
                board[3][5]=BOULDER;
                board[7][5]=BOULDER;
                for (i=0;i<COLS;i++) {
                        if (i%2==0)
                                board[5][i]=ROCK;
                }
                board[7][9]=HOLE;
                board[7][12]=HOLE;
                #endif
        }
        printBoard();
}

void movePlayer(int dx, int dy) {
    int newX = playerX + dx;
    int newY = playerY + dy;
    int destination = board[newX][newY];
    LED_Off(3);
    LED_Off(4);
    LED_Off(5);
    LED_Off(6);
    if (destination == EMPTY) { // Move to an empty space or target
        board[playerX][playerY] = EMPTY;
        playerX = newX;
        playerY = newY;
        board[playerX][playerY] = PLAYER;
        printBoard();
    }
    else if (destination == BOULDER) { // Push a boulder
        int boulderX = newX + dx;
        int boulderY = newY + dy;
        if (board[boulderX][boulderY] == EMPTY) {
            board[boulderX][boulderY] = BOULDER;
            board[newX][newY] = PLAYER;
```

```
            board[playerX][playerY] = EMPTY;
            playerX = newX;
            playerY = newY;
            printBoard();
        }
        else if (board[boulderX][boulderY] == HOLE) {
            board[boulderX][boulderY] = ROCK_IN;
            board[newX][newY] = PLAYER;
            board[playerX][playerY] = EMPTY;
            playerX = newX;
            playerY = newY;
            printBoard();
        }
        else if (board[boulderX][boulderY] == ROCK) {
            LED_On(3);
            GLCD_SetTextColor(White);
            GLCD_DisplayString(9, 0, __FI, "--Rock-in-the-way!--");
        }
        else if (board[boulderX][boulderY] == BOULDER) {
            LED_On(4);
            GLCD_SetTextColor(White);
            GLCD_DisplayString(9, 0, __FI, "Boulder-in-the-way!-");
        }
        else if (board[boulderX][boulderY] == ROCK_IN) {
            LED_On(5);
            GLCD_SetTextColor(White);
            GLCD_DisplayString(9, 0, __FI, "-Find-another-hole--");
        }
    }
    else if (destination == ROCK) {
        LED_On(3);
        GLCD_SetTextColor(White);
        GLCD_DisplayString(9, 0, __FI, "--Rock-in-the-way!--");
    }
    else if (destination == HOLE || destination == ROCK_IN) {
        LED_On(6);
        GLCD_SetTextColor(White);
        GLCD_DisplayString(9, 0, __FI, "Watch-out-for-holes!");
    }
}

int checkWin() {
   int i=0, j=0;
    for (i = 0; i < ROWS; i++) {
        for (j = 0; j < COLS; j++) {
            if (board[i][j] == HOLE || level == 0) {
                return 0; // Not all targets are covered
            }
        }
    }
    return 1; // All targets are covered
}

int game() {
```

```
        int KBD_but;
        level=0;
        // Start game
        GLCD_Clear(Black);
        GLCD_SetBackColor(Black);
        GLCD_SetTextColor(White);
        GLCD_DisplayString(0, 0, __FI, "----Sokoban-Game----");
        GLCD_DisplayString(2, 0, __FI, "Goal:-Push-all------");
        GLCD_DisplayString(3, 0, __FI, "boulders-into-the---");
        GLCD_DisplayString(4, 0, __FI, "holes----------------");
        GLCD_DisplayString(6, 0, __FI, "-----Game-Start-----");
for (;;){
        KBD_but = get_button ();
        switch(KBD_but){
                case (KBD_UP):
                        movePlayer(-1,0);
                        break;
                case (KBD_DOWN):
                        movePlayer(1,0);
                        break;
                case (KBD_LEFT):
                        movePlayer(0,-1);
                        break;
                case (KBD_RIGHT):
                        movePlayer(0,1);
                        break;
                case (KBD_SELECT):
                        GLCD_Clear(White);
                        initBoard();
                        break;
        }

        if (checkWin()) {
                if (level > 2) {
                        GLCD_DisplayString(5, 0, __FI, "------You-win!------");
                        GLCD_DisplayString(9, 0, __FI, "---Up->Main-Menu----");
                        switch(KBD_but) {
                                case (KBD_UP):
                                        GLCD_Clear(White);
                                        LED_Off(2);
                                        return 0;
                        }
                }
                else {
                        GLCD_DisplayString(6, 0, __FI, "---W's-in-the-chat--");
                        GLCD_DisplayString(9, 0, __FI, "---Up->Next-Level---");
                        switch(KBD_but) {
                                case (KBD_UP):
                                        level++;
                                        initBoard();
                                break;
                        }
                }
        }
```

```
        }
}
```

Listing 7: game.h

```
extern int game (void);
#define ROWS 12
#define COLS 16
//0=empty, 1=player, 2=boulder, 3=rock, 4=hole, 5=rock_in
extern int level1[ROWS][COLS] = {
        {0, 0, 0, 0, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 3, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 3, 2, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 3, 3, 3, 0, 0, 2, 3, 3, 3, 0, 0, 0, 0, 0},
        {0, 0, 3, 0, 0, 2, 0, 0, 2, 0, 3, 0, 3, 3, 3, 3},
        {3, 3, 3, 0, 3, 0, 3, 3, 3, 0, 3, 0, 3, 0, 0, 4},
        {3, 0, 0, 0, 3, 0, 3, 3, 3, 0, 3, 3, 4, 0, 0, 4},
        {3, 0, 2, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 4},
        {3, 3, 3, 3, 3, 3, 0, 3, 3, 3, 0, 3, 1, 0, 0, 4},
        {0, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 3, 3, 0, 0, 4},
        {0, 0, 0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
}       ;
extern int level2 [ROWS][COLS] = {
        {3, 3, 3, 3, 3, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0},
        {3, 0, 4, 4, 4, 4, 3, 3, 3, 3, 3, 3, 3, 0, 0, 0},
        {3, 4, 0, 0, 0, 0, 3, 0, 0, 0, 0, 0, 3, 3, 3, 0},
        {3, 4, 0, 0, 0, 0, 3, 0, 2, 0, 0, 2, 0, 0, 3, 0},
        {3, 4, 0, 0, 0, 0, 3, 2, 3, 3, 3, 3, 0, 0, 3, 0},
        {3, 4, 0, 0, 0, 0, 0, 0, 1, 0, 3, 3, 0, 0, 3, 0},
        {3, 0, 4, 4, 0, 0, 3, 0, 3, 0, 0, 2, 0, 0, 3, 0},
        {3, 3, 3, 3, 3, 3, 3, 0, 3, 3, 2, 0, 2, 0, 3, 0},
        {0, 0, 0, 3, 0, 2, 0, 0, 2, 0, 2, 0, 2, 0, 3, 0},
        {0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0},
        {0, 0, 0, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 0},
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
}       ;
```